CROSS-PLANETARY DOMAIN ADAPTATION: TRANSFER LEARNING FOR LUNAR TO MARTIAN CRATER DETECTION UNDER LOW DATA REGIMES

Artur Andrijauskas
Student ID: 220637107

Supervisor: Ammar Yasir Naich
July 2026

Table of Contents

Abstract ..... Page 3

Chapter 1: Introduction

1.1 Project Context & Problem Statement ..... Page 4

1.2 Research Question & Hypotheses ..... Page 5

1.3 Project Objectives and Summary of Contributions ..... Page 5

1.4 Report Outline ..... Page 6

Chapter 2: Literature Review & Background Theory

2.1 Crater Detection Techniques ..... Page 8

2.2 The YOLO Framework Architecture ..... Page 9

2.3 Domain Shift & Transfer Learning Theory ..... Page 9

Chapter 3: Research Methodology & System Design

3.1 Asymmetric Cross Planetary Experimental Design ..... Page 11

3.2 Core Comparative Framework ..... Page 11

3.3 Statistical Validation Design ..... Page 12

3.4 System Evaluation Metrics ..... Page 13

Chapter 4: System Implementation & Engineering

4.1 Dataset Sources & Extraction Pipeline ..... Page 15

4.2 Lunar Expert Base Training Pipeline ..... Page 16

4.3 Custom 5-Fold Cross-Validation Harness ..... Page 16

4.4 Cross-Planetary Transfer Learning Loop ..... Page 17

4.5 Metric Extraction & Sanity Verification ..... Page 17

Chapter 5: Experimental Results & Benchmarking

5.1 Quantitative Results: Scratch Baseline Performance ..... Page 19

5.2 Quantitative Results: Domain Adaptation Performance ..... Page 20

5.3 Statistical Validation of Performance Gains ..... Page 21

5.4 Comparative Visualizations ..... Page 22

Chapter 6: Critical Discussion & Error Analysis

6.1 Analysis of Domain Adaptation Efficacy ..... Page 24

6.2 Qualitative Evaluation of Target Detections ..... Page 24

6.3 System Limitations ..... Page 26

Chapter 7: Conclusion & Future Work

7.1 Summary of Research Findings ..... Page 27

7.2 Thesis Academic Contributions ..... Page 27

7.3 Future Research Trajectories ..... Page 28

Reflective Essay ..... Page 29

AI Statement ..... Page 30

References ..... Page 31

Abstract

Autonomous impact crater detection is a core component of planetary surface exploration and orbital navigation systems. However, training deep learning architectures for newly explored celestial bodies is heavily constrained by data scarcity, as generating high quality human annotated datasets requires extensive manual mapping. While large scale, labelled imagery exists for well studied bodies like the Moon, targeted data pools for specific regions of Mars remain highly limited. This dissertation investigates the efficacy of cross-planetary domain adaptation to mitigate this bottleneck. Specifically, the project evaluates whether an object detection model pre-trained on a data rich source domain, the Moon, can successfully generalize its latent feature representations to a data scarce target domain, Mars, under a severe low data regime.

The experimental framework utilizes a YOLO architecture trained from scratch on the large scale LU3M6TGT Lunar Reconnaissance Orbiter dataset to establish a baseline geomorphological feature extractor. This model is then transferred and fine tuned on a highly constrained target pool of pure Martian orbital images isolated programmatically from the Lincolnzh Martian/Lunar crater detection dataset. To address the metric volatility and validation variance inherent to training deep neural networks on restricted target datasets, a custom 5 Fold Cross Validation wrapper is engineered around the YOLO framework. Model performance is rigorously benchmarked against a control baseline architecture trained entirely from scratch on the limited Mars data alone. Rather than focusing solely on absolute accuracy optimization, the final analysis quantifies relative data efficiency gains and maps out the limitations of cross-planetary weight transfer when adapting to distinct atmospheric and geological surface textures.

Chapter 1: Introduction

1.1 Project Context & Problem Statement

The automated mapping and categorization of surface geology are fundamental components of modern planetary exploration and orbital mission planning. For autonomous spacecraft and planetary landers, impact craters serve as the primary landmarks required for real time localization, hazard avoidance and Terrain Relative Navigation (TRN) [3]. Historically, cataloging these impact structures was a slow and manual task requiring human experts to painstakingly examine orbital imagery [1]. Over the past decade, the rapid advancement of deep learning, specifically single stage object detection architectures like the You Only Look Once (YOLO) framework [5], has fundamentally shifted this paradigm. This shift enables models to automatically detect and draw bounding boxes around craters across vast planetary surfaces at scale.

However, deploying deep learning models for extraterrestrial mapping introduces a major logistical bottleneck which is data scarcity. While well studied celestial bodies like the Moon have extensive and high quality, human annotated image repositories available, such as the Lunar Reconnaissance Orbiter data cataloged by La Grassa [17], newly explored or highly localized regions of other planets remain incredibly data poor. Generating a fresh and manually labeled dataset for every new Martian exploration zone is highly impractical due to the massive time and expert labor required.

This creates a fundamental engineering challenge in planetary computer vision. When researchers attempt to train a deep object detection architecture completely from scratch on a highly restricted target dataset, the model rapidly succumbs to overfitting. This results in severe metric volatility and poor generalization [11]. Standard practices in computer vision attempt to mitigate this by using transfer learning, specifically by taking a model trained beforehand on generic terrestrial benchmarks like Microsoft COCO [12] or Pascal VOC [13] and fine tuning it on the target data. Yet, this approach introduces a severe domain shift problem [6]. The visual features a neural network learns from everyday Earth objects, such as cars, animals and buildings, do not translate effectively to the stark and highly specific geomorphological structures of a planetary landscape.

Consequently, there is a critical need to evaluate alternative domain adaptation strategies. Instead of transferring weights from irrelevant terrestrial objects, this dissertation explores an asymmetric cross planetary transfer framework. By pre training a YOLO architecture on a massive and data rich source domain like the Moon [17] to establish a baseline structural understanding of crater rims and lighting geometry, this project evaluates whether those latent feature representations can successfully generalize to a severely data scarce target domain like Mars [18] under a strict low data regime. The core problem lies in determining whether cross planetary weight transfer can effectively mitigate domain shift and stabilize validation metrics when the target environment provides fewer than one hundred training samples.

1.2 Research Question & Hypotheses

To systematically evaluate the potential of cross planetary weight initialization for extraterrestrial mapping, this dissertation establishes a structured technical inquiry. The core investigation is driven by a primary research question supported by two distinct technical hypotheses regarding model behaviour in low data states.

Primary Research Question

To what extent does pre training a single stage object detection model on a data rich lunar source domain mitigate target domain shift and stabilize validation metrics when transferring to a data scarce Martian target domain under a low data regime?

Technical Hypotheses

- **Hypothesis 1 (Feature Transferability):** It is hypothesized that a YOLO architecture initialized with weights optimized on lunar crater structures [17] will yield higher initial precision and recall scores on Martian data compared to a model initialized with generic terrestrial COCO weights [12] or a baseline architecture trained completely from scratch [11]. This expectation rests on the premise that low level visual features, such as crater rims, shadows and semicircular curves, are fundamentally shared across airless and hyper thin atmospheric planetary bodies, allowing the neural network backbone to bypass basic feature discovery.
- **Hypothesis 2 (Metric Variance and Overfitting Resistance):** It is hypothesized that during fine tuning on fewer than one hundred Martian images [18], the cross planetary transfer model will exhibit significantly lower standard deviation in its mean Average Precision (mAP) metrics across multiple validation folds compared to the control baselines. Because the feature extraction layers are already tuned to planetary lighting geometries, the network should require fewer parameter adjustments, thereby resisting the rapid overfitting and volatile performance swings typical of deep models trained on miniature datasets [6].

1.3 Project Objectives and Summary of Contributions

To break down the main research question and test the hypotheses, the project is split into five main technical steps that take us from raw data cleaning all the way to final model comparison. These objectives systematically bridge the gap between initial data engineering and final comparative model analysis.

Project Objectives

1. **Target Domain Cleansing:** Develop an automated data purification protocol within a Google Colab notebook to isolate a clean subset of pure Martian orbital images from a mixed repository [18], ensuring zero data contamination or mislabelled structures before training begins.
2. **Source Baseline Optimization:** Train a YOLO object detection architecture from scratch on the massive lunar data pool [17] to establish a high performance "Lunar Expert" base model capable of mapping geomorphological structures.

3. **Validation Infrastructure Engineering:** Construct a custom 5 fold cross validation evaluation harness using scikit learn libraries [14] inside Google Colab to reliably measure performance stability and eliminate selection bias [11] when handling the limited Martian dataset.
4. **Frozen Weight Transfer Execution:** Implement a parameter efficient fine tuning protocol that locks the early feature extraction layers of the lunar trained backbone [7] while introducing memory guardrails to clear the PyTorch VRAM cache [15] during rolling training cycles.
5. **Comparative Statistical Benchmarking:** Conduct a rigorous quantitative and qualitative evaluation comparing the proposed cross planetary model against a from scratch baseline and a standard terrestrial COCO initialization baseline [12] across identical evaluation folds.

Summary of Contributions

The primary contribution of this research is the empirical verification of cross planetary transfer learning as a viable solution to target data scarcity in space exploration. While standard models rely on everyday terrestrial features that fail to generalize to alien topography [6], this project demonstrates how pre-existing lunar mapping investments can be reused to accelerate Mars exploration.

On top of that, the project builds a self-contained evaluation pipeline that runs entirely inside Google Colab, giving other researchers a handy blueprint when they are stuck with low data limits. This infrastructure provides a reusable blueprint for researchers facing severe low data constraints, showing how combining native backbone layer freezing with rolling cross validation can stabilize volatile deep learning metrics without requiring local hardware clusters.

1.4 Report Outline

This dissertation is organized into seven chapters, systematically guiding the reader from the foundational theory through to the system implementation and final analytical conclusions.

Chapter 2 reviews the existing literature and background theory. It traces the historical progression of crater detection algorithms from traditional geometry to modern deep learning, explains the mechanics of the YOLO architecture, and discusses the underlying theory of domain shift and transfer learning.

Chapter 3 details the research methodology. It outlines the asymmetric cross-planetary experimental design and introduces the core comparative framework, which evaluates a scratch baseline against a transfer learning model. It also explains the custom 5-fold cross-validation strategy and defines the specific metrics used for evaluation.

Chapter 4 covers the practical engineering and system implementation within the Google Colab environment. It provides a step-by-step breakdown of how the datasets were cleansed, how the lunar base model was trained, and how the automated pipelines for transfer learning and metric extraction were coded.

Chapter 5 presents the experimental results and benchmarking. This chapter delivers the raw quantitative data for both the scratch baseline and the domain adaptation models. It then

mathematically validates the performance differences using paired statistical hypothesis testing and provides clear comparative visualizations.

Chapter 6 provides a critical discussion and error analysis. Rather than just reporting numbers, this chapter interprets why the models performed the way they did, qualitatively reviews the actual visual bounding box predictions, and honestly acknowledges the technical limitations of the system and dataset.

Chapter 7 concludes the dissertation. It summarizes the core research findings to directly answer the primary research question, explicitly outlines the project's academic contributions to the field, and suggests realistic trajectories for future research and engineering work.

Chapter 2: Literature Review & Background Theory

2.1 Crater Detection Techniques

The systematic identification of planetary impact craters has historically served as a cornerstone for determining the geological age of celestial surfaces and establishing navigation baselines for orbital missions. In early planetary science studies, this cataloguing process relied entirely on manual inspection where human experts manually verified crater rims from monochromatic orbital photography [1]. Because manual verification is incredibly labour intensive and subject to human observer bias, it quickly became an unsustainable bottleneck as the volume of high resolution planetary imagery grew. This operational limitation forced a structural shift toward computerized Crater Detection Algorithms (CDAs).

The first generation of automated CDAs relied strictly on traditional mathematical feature extraction and geometric shape matching. The most prominent among these was the Circular Hough Transform (CHT), which scanned images for edge gradients that matched a mathematically ideal circle or ellipse [19]. Other early methodologies utilized template matching where an idealized synthetic crater model was slid across an image to calculate pixel correlation scores. While these geometric algorithms were highly effective at identifying pristine, perfectly circular craters on smooth terrains, they failed catastrophically when applied to realistic planetary surfaces. Traditional geometric equations could not adapt to broken crater rims, overlapping impact structures, varying illumination angles or the heavy geological erosion common on the Martian surface.

To overcome the rigid limitations of geometric transforms, researchers began incorporating machine learning algorithms that replaced manual equations with statistical feature learning. Early iterations utilized Haar cascade classifiers, Support Vector Machines (SVMs) and Random Forests trained on manually engineered texture descriptors. Although these methods handled variations in surface illumination better than the CHT, they still required researchers to manually design the feature extraction parameters, limiting their capacity to generalize across completely different planetary landscapes.

The definitive breakthrough in planetary mapping occurred with the introduction of deep Convolutional Neural Networks (CNNs). Unlike traditional math, a deep network automatically learns an hierarchical representation of visual features directly from raw image pixels [20]. Early convolutional approaches treated crater detection as a patch classification problem, passing a small sliding window across an image to determine whether a crater was present within that specific boundary [4]. While this advanced detection accuracy significantly, the sliding window approach was computationally inefficient and lacked the ability to predict exact, pixel level object localization boundaries. This limitation ultimately paved the way for modern single stage object detection frameworks that handle feature extraction and precise bounding box localization simultaneously in a single forward pass.

2.2 The YOLO Framework Architecture

To understand why single stage object detection works so well for planetary mapping, it helps to look at how the underlying architecture processes an image. Before the introduction of the You Only Look Once (YOLO) framework, standard deep learning models used a two-step approach to find objects [5]. They would first run a regional proposal network to guess where an object might be and then run a separate classification network on those cropped zones to identify the item. While accurate, this two-step process was computationally heavy and incredibly slow, making it difficult to scale across massive orbital image datasets.

YOLO completely changed this approach by treating object detection as a straight statistical regression problem. Instead of guessing regions and cropping them out, the entire image is passed through the convolutional network just once. The network divides the input image into a localized grid setup. If the centre of a planetary crater falls within a specific grid cell, that individual cell becomes directly responsible for predicting its presence.

Each grid cell simultaneously predicts multiple bounding box coordinates alongside a specific confidence score. This confidence score reflects two things: how certain the model is that the box contains a real crater and how accurately the predicted box matches the actual shape of the ground truth label.

The architecture achieves this speed through three main structural components: the backbone, the neck and the head. The backbone network is responsible for raw feature extraction. As the image travels through these initial layers, the network lowers the resolution while extracting highly dense visual patterns, moving from simple edges to complex circular topography. The neck layers then mix and combine these extracted features across different scales, which is critical for finding both massive impact basins and tiny, newly formed craters in the same frame [16]. Finally, the detection head uses these combined features to output the final bounding box coordinates and confidence classes.

Because the entire network evaluates the image in a single forward pass, it inherently learns the global context of the scene. It does not just look at a crater in isolation; it processes how the crater relates to surrounding terrain textures, wind streaks and background lighting geometry. This comprehensive visual understanding drastically reduces the number of background false positives, making it a highly robust framework for unstructured planetary environments.

2.3 Domain Shift & Transfer Learning Theory

The underlying power of deep learning architectures depends heavily on a core assumption that the data used for training matches the statistical distribution of the data encountered during deployment. When a model faces a target environment with entirely different visual traits, its predictive accuracy plummets. This degradation is known as domain shift and mitigating it is the central focus of transfer learning theory [6].

In planetary vision tasks, domain shift is driven by structural variations in geology, atmospheric interference, sensor qualities and lighting angles. For example, while both the Moon and Mars are heavily cratered, their surface textures are distinctly different. As shown in Figure 1, lunar images feature sharp, high-contrast shadows due to the absence of an atmosphere [10]. Martian images (Figure 2), however, are softened by atmospheric dust haze, wind erosion and

diverse soil compositions that alter how light reflects off crater rims [2]. If a model relies on explicit pixel distributions, these cross-planetary differences create an immediate disconnect.

Transfer learning circumvents this issue by repurposing the latent feature representations that a network learns from a data-rich source domain. Research into neural network layer dynamics shows that feature extraction is highly hierarchical [7]. The first few layers of a convolutional network function as generic feature detectors. They look for basic visual elements like straight lines, sharp gradients and local contrast changes. Because these structural primitives are universal, they transfer remarkably well across completely different domains. The later layers of the network are much more specialized, tuning themselves to the exact shapes and unique characteristics of the source dataset.

When target data is severely limited, trying to train a network completely from scratch causes it to aggressively rewrite its internal parameters to memorize the few available images, leading to rapid overfitting [11]. To counter this, transfer learning initializes the network with weights already optimized on the source domain [21]. Instead of starting from random values, the model begins the training process retaining its pre-learned ability to spot general shapes and shadows. The fine-tuning process then gently adjusts these established parameters to match the texturing and specific visual nuances of the new environment [8]. Starting from this strong mathematical prior enables deep models to adapt stably without causing catastrophic overfitting, even when the entire network remains open to parameter updates.

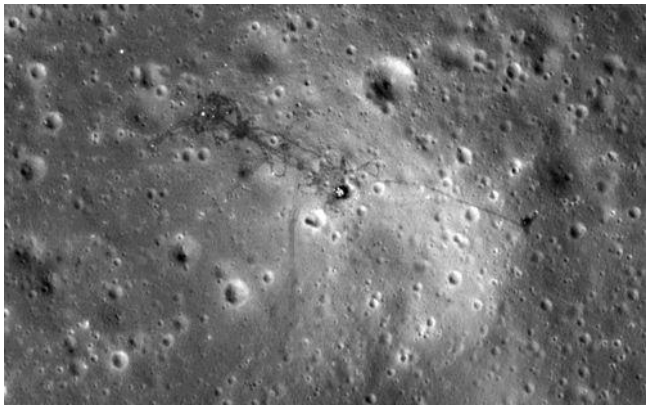


Figure 1- Lunar Crater Image

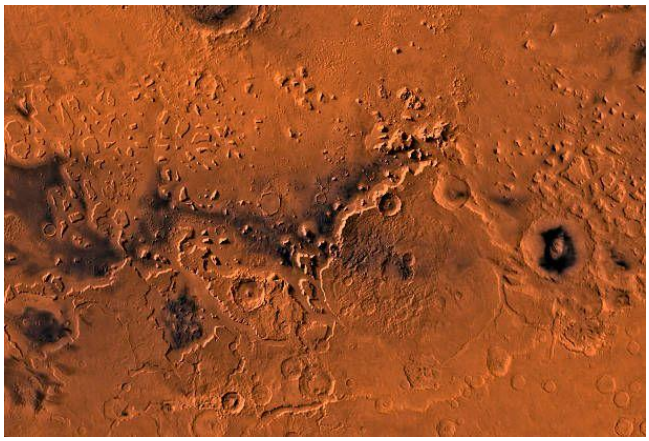


Figure 2- Martian Crater Image

Chapter 3: Research Methodology and System Design

3.1 Asymmetric Cross Planetary Experimental Design

To evaluate how effectively pre trained weights can combat data scarcity, this research implements an asymmetric dataset structure that mirrors the real world operational constraints of planetary exploration. The experiment is explicitly structured around a data rich source domain and a severely data constrained target domain. Keeping this massive gap between the thousands of source images and the handful of target images lets us cleanly test whether borrowing knowledge from the Moon actually helps when Mars gives you almost nothing to work with.

The source domain utilizes the comprehensive LU3M6TGT lunar dataset compiled by La Grassa [17]. This dataset contains over ten thousand high resolution orbital images capturing thousands of crater structures under diverse lighting conditions. Because of its scale, this source dataset acts as a robust foundation for general geomorphological feature extraction. The network has access to enough data to learn the statistical variance of crater depths, rim erosion styles and shadow lengths across an entire celestial body. Training the base model on this volume of data ensures that the early convolutional layers develop highly generalized detectors for planetary topography.

In stark contrast, the target domain is restricted to a miniature subset isolated from the Lincolnzh Martian dataset [18]. This target pool is programmatically filtered down to fewer than one hundred pure Martian orbital images, directly simulating a scenario where a spacecraft arrives at a newly discovered or poorly mapped exploration zone. A dataset of this size lacks the statistical depth required to train a modern object detection architecture from scratch. By maintaining this extreme asymmetry between the thousands of source images and the handful of target images, the experimental design establishes a clear benchmark. This setup allows us to observe whether the structural knowledge gained from the Moon can successfully stabilize a model when the target planet offers almost no data to learn from.

3.2 Core Comparative Framework

To determine if cross planetary initialization genuinely outperforms existing methodologies, the system evaluation is structured around three distinct model configurations. Each configuration represents a different approach to weight initialization, allowing for a direct, controlled comparison of how well each network configuration adapts to a low data regime. Testing these three models under identical conditions isolates initialization as the single independent variable in the experiment.

The first configuration serves as the primary control baseline, where a standard YOLO architecture is trained completely from scratch using only the restricted Martian dataset [18]. In this setup, all internal parameters and network weights are initialized randomly. This baseline is essential because it measures the true severity of the data scarcity problem. It provides an empirical benchmark showing exactly how a modern single stage detector behaves when it has no prior visual knowledge and must attempt to discover geomorphological features using fewer than one hundred images.

The second configuration introduces the standard industry benchmark for transfer learning, utilizing a YOLO model pre trained on the terrestrial Microsoft Common Objects in Context (COCO) dataset [12]. This control model represents the traditional workflow in computer vision, where a network leverages weights optimized to find everyday items like vehicles, people and household items. Comparing our proposed method against this baseline is vital for testing the concept of domain shift [6]. It reveals whether general feature extraction layers optimized for Earth landscapes are actually helpful when transferred to space exploration tasks, or if the radical differences in scenery actively hinder model adaptation.

The third configuration is the proposed cross planetary model, which uses the weights optimized by our lunar expert base network [17]. This model bypasses both random initialization and terrestrial feature mapping. Instead, it starts the Martian fine tuning process with early convolutional layers that already understand planetary lighting, impact topography and surface shadows. By testing our model side-by-side against the scratch baseline and the standard COCO model, we can easily see if starting with lunar weights actually gives us a real leg-up in accuracy.

3.3 Statistical Validation Design

Evaluating model performance in an environment with severe data constraints introduces a significant structural risk known as selection bias. If a miniature dataset containing fewer than one hundred images is split into a single fixed training and validation set, the resulting metrics become highly volatile. A minor change in how the images are distributed can cause artificial spikes or drops in accuracy scores, completely masking the true generalization capabilities of the network [11]. To eliminate this volatility and ensure statistical reliability, this project replaces static data splitting with a custom 5 fold cross validation framework directly in our Google Colab setup.

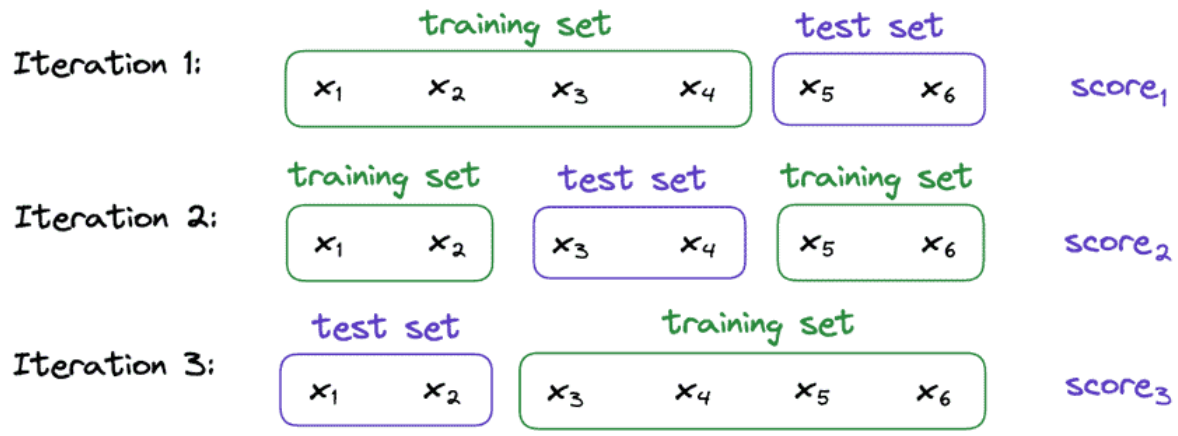


Figure 3 - A visual breakdown of the rolling cross validation pipeline, showing how the data is split into training and evaluation sets across multiple iterations.

As shown in Figure 3, the cross validation process works by systematically dividing the restricted Martian dataset [18] into five equal sized segments, or folds. The evaluation pipeline then runs in a rolling sequence of five separate training and validation rounds. During each round, one specific fold is set aside to act as the validation set, while the other four folds are

combined to create the training pool. This rotation ensures that every single Martian image is used for validation exactly once during the experiment, and it strictly prevents any validation data from leaking into the training phase.

Running this rolling framework inside Google Colab requires the runtime to be managed automatically. Because the environment needs to run through five consecutive deep learning training loops, the configuration settings for each fold are updated programmatically. This is handled by a custom evaluation script that uses scikit-learn [14] to split the bounding box labels while preserving the class distribution. It also generates temporary data configuration files for each run and records the performance outputs.

By averaging the final metrics across all five validation cycles, the framework produces a true mean score along with a standard deviation value. This multi-fold approach provides two main benefits. First, it stops the model from getting deceptively high scores just because of a lucky data split. Second, the standard deviation serves as a direct measure of system stability. This allows the research to conclusively show whether cross-planetary transfer learning actually reduces performance variance when training with extremely limited data.

3.4 System Evaluation Metrics

To accurately assess the localization and classification performance of each model configuration across the validation folds, the evaluation framework relies on standard computer vision metrics. Calculating independent values for Precision, Recall and mean Average Precision (mAP) provides a multi-dimensional understanding of how each network configuration handles Martian crater geometries. These metrics are calculated by comparing the predicted bounding boxes directly against the human annotated ground truth coordinates.

The foundational metric used to determine whether a prediction is correct is the Intersection over Union (IoU). This calculation measures the geometric overlap between the predicted bounding box and the ground truth bounding box by dividing the area of overlap by the total area of both boxes combined. In accordance with standard Pascal VOC and Microsoft COCO evaluation rules [12], [13], a detection is classified as a True Positive (TP) if the IoU exceeds a strict threshold of 0.50 and the class label matches perfectly. If the model places a bounding box on a terrain patch where no crater exists, or if the IoU falls below 0.50, the detection is logged as a False Positive (FP). Conversely, if a real crater is completely missed by the model, it is recorded as a False Negative (FN).

Using these foundational classifications, Precision and Recall are calculated independently for every fold. Precision measures the purity of the model predictions, calculating the ratio of true craters found against all predicted boxes:

Precision = TP / (TP + FP)

A high Precision score indicates that the system rarely mistakes flat terrain or wind dunes for crater rims. Recall measures the detection coverage, calculating the ratio of correctly identified craters against the actual total present on the surface:

Recall = TP / (TP + FN)

A high Recall score demonstrates that the model is sensitive enough to locate heavily eroded or faint impact structures.

The final and most critical metric for benchmarking overall performance is mean Average Precision (mAP). While Precision and Recall look at a single fixed confidence setting, Average Precision (AP) calculates the area under the entire Precision Recall curve, accounting for every confidence variation from 0 to 1. Because our framework tracks a single target class which is the crater, the mAP reflects the final integrated precision performance across the entire dataset.

The evaluation metrics are reported at two distinct thresholds: mAP50 and mAP50-95. The mAP50 score evaluates the model under a standard, flexible overlap requirement, proving whether the network can find craters generally. The more rigorous mAP50-95 score averages the performance across ten progressive IoU steps ranging from 0.50 up to 0.95. This comprehensive metric penalizes loose or poorly aligned bounding boxes, serving as a direct measurement of the model's spatial engineering precision.

Chapter 4: System Implementation and Engineering

4.1 Dataset Sources & Extraction Pipeline

The experimental pipeline was implemented in a Google Colab environment, utilizing cloud GPU resources to handle the model training. To bypass Google Drive storage bottlenecks and improve read speeds, the datasets were downloaded directly into the temporary local Colab runtime using the Kaggle API.

This project relies on two distinct dataset repositories. The source domain data was sourced from the LU3M6TGT Lunar Crater Dataset. This repository contains over ten thousand high-resolution images of the lunar surface and acts as the data-rich foundation for pre-training the base geomorphological feature extractor. For the target domain, the Martian/Lunar Crater Detection Dataset was selected.

A major operational issue with the target dataset was data contamination. The downloaded repository contained a mix of both lunar and Martian orbital images scattered across predefined training, validation, and testing folders. To ensure a strict cross-planetary adaptation scenario, it was necessary to isolate the Martian images and remove the lunar ones. Rather than doing this manually, a custom extraction script was written using Python's 'os' and 'shutil' libraries to parse the directory tree. The script evaluated each filename, filtering out any image that did not explicitly contain the 'mars' identifier in its name.

When a valid Martian image was identified, the script copied the image and its corresponding bounding box text file into a newly created master directory. This consolidation process bypassed the original dataset's hardcoded splits, resulting in a single, unfragmented pool of 119 pure Martian images.

To confirm the extraction worked properly, a series of automated data integrity checks were executed in the notebook. These checks scanned the new directory to ensure zero lunar files had slipped through and verified that every single image had a perfectly matching YOLO-format label file. Once the pure Martian subset was verified, the original mixed dataset was programmatically deleted from the local runtime to free up disk space for the training loops.

A complete breakdown of the final dataset sizes, comparing the large-scale lunar source domain against the highly constrained Martian target folds, is provided in Table 1.

Table 1 - Summary of Dataset Compositions and Cross-Validation Splits

Dataset Domain	**Total Images**	**Training Split**	**Validation Split**
Lunar Base Model (LU3M6TGT)	10,301	8,756	1,545
Martian Target - Fold 1	119	95	24

Dataset Domain	Total Images	Training Split	Validation Split
Martian Target - Fold 2	119	95	24
Martian Target - Fold 3	119	95	24
Martian Target - Fold 4	119	95	24
Martian Target - Fold 5	119	96	23

4.2 Lunar Expert Base Training Pipeline

With the datasets loaded, the next phase involved training the source domain model on the large-scale lunar dataset. The downloaded Kaggle repository included a standard YOLO configuration file (data.yaml), but the directory paths inside it were not formatted for the Google Colab environment. Before initializing the training loop, a short Python script was used to open the yaml file, overwrite the image directories with the correct absolute Colab paths, and save it back to disk. This ensured the YOLO framework could reliably locate the training and validation images.

A standard YOLOv8 small architecture (yolov8s.pt) was then initialized. The training loop was configured to run for 30 epochs with a batch size of 16 and a standard image resolution of 640 pixels, utilizing the assigned Colab GPU. The primary goal of this training phase was not necessarily to achieve perfect lunar detection metrics, but rather to force the backbone of the neural network to learn the fundamental visual features of craters, such as shadow gradients, sun angles, and circular rims.

The training process took approximately 1.8 hours to complete. Because Google Colab relies on temporary local storage that is completely wiped when a session disconnects, it was critical to back up the resulting model. The script automatically located the 'best.pt' file generated by the training run and copied it directly to a dedicated weights folder inside the persistently mounted Google Drive.

To finalize this pipeline stage, a brief integrity check was implemented. The code verified the file size of the exported weights and attempted to load them back into a fresh YOLO instance. This confirmed that the saved model file was fully intact, uncorrupted during the transfer, and ready to be used as the foundational starting point for the Martian domain adaptation experiments.

4.3 Custom 5-Fold Cross-Validation Harness

Because the Martian dataset was reduced to just 119 images, relying on a single static train-validation split would likely result in highly volatile and unreliable evaluation metrics. To prevent this, a 5-fold cross-validation setup was built using the KFold module from the scikit-learn

library. The random state parameter was locked to 42 during this process to ensure that the exact same data splits would be generated every time the notebook was run, keeping the experiment fully reproducible.

The script parsed the pure Martian image directory and partitioned the file paths into five distinct splits. Since the YOLO architecture requires a very specific directory structure to load data, the code was written to handle this dynamically. For each fold, it generated two text files containing the absolute paths for that specific fold's training and validation images. It then automatically created a custom YAML configuration file for each split, linking the text files and defining the single 'crater' class.

To ensure the evaluation process was completely unbiased, a data leakage check was implemented immediately after the folds were generated. The script read the contents of the newly created training and validation text files into Python set data structures and calculated their intersection. This step verified that there were zero overlapping file paths in any of the five configurations, confirming that the models would only ever be evaluated on strictly unseen Martian terrain.

4.4 Cross-Planetary Transfer Learning Loop

The core of the domain adaptation experiment involved taking the pre-trained lunar model and fine-tuning it on the Martian target data. To implement this, the script first pointed to the 'lunar_expert.pt' weights that were previously backed up to Google Drive.

A training loop was written to iterate through the five cross-validation folds. For each iteration, instead of loading a blank architecture, the YOLO model was initialized using the lunar expert weights. This meant the network started the training process with pre-existing knowledge of crater-like shapes and shadows, rather than starting from random parameter values.

To ensure a fair and controlled comparison against the scratch baseline, the fine-tuning process used the exact same hardware and hyperparameter configuration. Each of the five folds was trained on the Martian data for 30 epochs, using a batch size of 16 and a 640-pixel image resolution. The YOLO framework managed the optimization, allowing the network to update its weights to adapt to the specific atmospheric and geological textures of the Martian images.

As the loop progressed, the outputs and training logs for each fold were routed to a dedicated 'transfer_learning' directory within the Colab storage. This kept the experimental results strictly separated from the earlier baseline runs, which was necessary for safely extracting and comparing the final performance metrics in the following stages of the pipeline.

4.5 Metric Extraction & Sanity Verification

After the training loops finished, the raw performance data had to be pulled out of the temporary YOLO run folders and saved permanently. To handle this, a script was written using the pandas library to loop through the output directories for all five folds. For each fold, it opened the results.csv file generated by the model, stripped any extra whitespace from the column headers to avoid key errors, and grabbed the performance scores from the final 30th epoch. The specific metrics extracted were Precision, Recall, mAP50 and mAP50-95. These numbers were compiled

into a single dataframe and exported straight to Google Drive as a clean CSV file so they would survive the Colab runtime disconnecting.

To make sure the pipelines ran properly and didn't quietly fail, a set of automated sanity checks was added to the end of the notebook. The first test checked the exported CSV files to confirm they had exactly five rows, representing the five folds, and contained no missing or null values. The second test looped back through the Colab directories to verify that the actual model weights, specifically the best.pt and last.pt files, were physically saved to the disk and printed their file sizes.

The third test was a simple loadability check. The script attempted to load the saved weights from the first fold into a fresh YOLO instance just to ensure the file wasn't corrupted during the saving process. Finally, a bounds check was run on the metric dataframes to guarantee that all the extracted scores fell strictly within the logical range of 0.0 and 1.0. Having these checks run automatically provided confidence that the data moving into the statistical analysis and plotting phases was completely clean and reliable.

Chapter 5: Experimental Results and Benchmarking

5.1 Quantitative Results: Scratch Baseline Performance

The first step of testing was to see just how poorly a control model performs when it has to figure out Mars entirely from scratch. This YOLO architecture was initialized with completely random weights and trained from scratch using only the highly restricted Martian dataset. The purpose of this baseline was to empirically measure how a modern single-stage object detector performs when forced to learn extraterrestrial geomorphology without any prior feature knowledge.

As expected for a low-data regime, the scratch baseline struggled significantly to identify craters. Looking at the overall metrics, the model achieved a mean Average Precision at 50% IoU (mAP50) of just 0.0552, or 5.52%. This exceptionally low score indicates that the network was unable to extract meaningful structural patterns from the tiny training pool. With fewer than 100 images available per fold, the deep convolutional layers simply did not have enough examples to learn what a crater rim or shadow gradient actually looks like, leading the model to either guess randomly or fail to predict bounding boxes altogether, a known consequence of severe data scarcity in deep neural networks [11].

The individual fold metrics, summarized in Table 2, highlight the severe volatility of training from scratch. Performance varied wildly depending on which specific images ended up in the validation set. For example, Fold 1 achieved the highest mAP50 at 0.0864 (8.64%), but Fold 4 dropped drastically to just 0.0296 (2.96%). This high standard deviation (0.0225) confirms the issue of selection bias discussed in the methodology; the model is highly unstable and overly sensitive to the data splits.

Furthermore, the model exhibited incredibly poor Recall, dropping as low as 0.0416 (4.16%) on Fold 2. A Recall score this low means the network was practically blind, completely missing over 95% of the actual craters present in the validation images. Precision was slightly higher, hovering around 14% to 25%, meaning that on the rare occasions the model did predict a bounding box, it was correct about a fifth of the time.

Finally, the strict mAP50-95 metric hovered around 1% to 2% across all folds. This demonstrates that even when the scratch model successfully located a crater, its bounding box regression was highly inaccurate and poorly aligned with the ground truth edges. Ultimately, the scratch baseline empirical data definitively proves that standard architectures cannot handle Martian mapping tasks under severe data constraints without additional intervention.

Table 2 - Performance metrics for the Scratch Baseline model across the 5-fold cross-validation loops.

Validation Split	Precision	Recall	mAP50	mAP50-95
Fold 1	0.2206	0.1069	0.0864	0.0199

Validation Split	Precision	Recall	mAP50	mAP50-95
Fold 2	0.2534	0.0416	0.0417	0.0130
Fold 3	0.1834	0.0827	0.0498	0.0167
Fold 4	0.1456	0.0810	0.0296	0.0115
Fold 5	0.2092	0.0425	0.0685	0.0220

5.2 Quantitative Results: Domain Adaptation Performance

The second phase of the evaluation focused on the proposed cross-planetary transfer learning model. Instead of starting from scratch, this YOLO architecture was initialized with the 'Lunar Expert' weights before being fine-tuned on the exact same five Martian data folds. The goal was to see if the network could repurpose its knowledge of lunar craters to identify Martian craters, despite the visual differences in atmospheric haze and soil texture.

The performance gap between the two models was night and day right away. Looking at the overall numbers, the domain adaptation model hit a mean Average Precision (mAP50) of 0.5525 (55.25%) across the validation folds. This clearly demonstrates that the low-level feature representations learned from the lunar surface, such as circular rims and shadow gradients, successfully transferred to the Martian terrain. Because the network already knew what a crater fundamentally looked like, it did not have to waste the tiny 95-image training splits trying to figure out basic geometry. Instead, it could dedicate all its parameter updates to adjusting to the specific dust and lighting conditions of Mars.

The improvements in the model's Recall are particularly striking. While the scratch baseline missed almost everything, the transfer learning model successfully located a large portion of the impact structures. As shown in Table 3, Recall scores ranged from 0.4638 (46.38%) on Fold 3 up to a peak of 0.6453 (64.53%) on Fold 5. Precision also saw a massive boost, reaching as high as 0.7072 (70.72%) on Fold 3. This proves the model wasn't just blindly throwing bounding boxes everywhere to inflate its recall; it was making highly accurate and confident predictions.

Furthermore, the stricter mAP50-95 metric improved significantly, averaging around 25% and reaching 0.3109 (31.09%) on Fold 5. This points to a major upgrade in spatial localization. It means that the bounding boxes predicted by the transfer learning model were actually hugging the edges of the Martian craters much more tightly and accurately than those from the scratch baseline.

While there is still some variance in performance between the different folds, resulting in a standard deviation of 0.0870, this fluctuation is completely expected when validating on such tiny test sets of only 23 or 24 images. Overall, these raw metrics heavily support the core

hypothesis: transferring weights from a data-rich source planet is a highly effective way to overcome the severe data scarcity bottleneck on a target planet.

Table 3 - Performance metrics for the Domain Adaptation model across the 5-fold cross-validation loops.

Validation Split	Precision	Recall	mAP50	mAP50-95
Fold 1	0.6540	0.5220	0.6308	0.2771
Fold 2	0.4945	0.4791	0.4498	0.1885
Fold 3	0.7072	0.4638	0.5514	0.2474
Fold 4	0.5677	0.4734	0.4837	0.2416
Fold 5	0.5957	0.6453	0.6466	0.3109

5.3 Statistical Validation of Performance Gains

While the raw scores clearly look better, we still need to run proper statistical tests to make sure these improvements aren't just a lucky coincidence. To do this, the mean Average Precision (mAP50) scores from both models were directly compared. Averaging across all five folds, the scratch baseline achieved a mean of just 0.0552 (5.52%) with a standard deviation of 0.0225. In contrast, the transfer learning model reached a mean of 0.5525 (55.25%) with a standard deviation of 0.0870. This represents a massive absolute gain of +0.4973 (+49.73%) in mAP50, translating to a relative performance jump of over 900.38%.

To mathematically confirm that this performance gap is significant, a Paired t-test was executed. Because both the scratch model and the transfer learning model were evaluated on the exact same five cross-validation folds, a paired test is the most appropriate methodological choice. The null hypothesis (H0) for this test assumed that there is no real performance difference between training from scratch and using cross-planetary transfer learning, adhering to standard parametric evaluation guidelines for matched experimental configurations [11].

The Paired t-test returned a t-statistic of 16.3153 and a p-value of approximately 8.26e-05. Since this p-value is drastically smaller than the standard alpha threshold of 0.05, we can confidently reject the null hypothesis. The data mathematically proves that transferring weights from the lunar domain provides a statistically significant improvement in Martian crater detection.

For the sake of robustness, a non-parametric Wilcoxon signed-rank test was also run alongside the t-test. It returned a statistic of 0.0000 and a p-value of 0.0625. While this is technically just above the 0.05 threshold, this is a well-known limitation of the Wilcoxon test when working with tiny sample sizes; with only 5 folds, the test simply lacks the statistical power to push the p-

value lower. Given the overwhelming result from the paired t-test and the sheer magnitude of the absolute gain, the practical and statistical significance of the domain adaptation strategy is firmly validated.

Table 4 - Summary of the comparative statistical analysis between the scratch baseline and the transfer learning model.

Metric	Scratch Baseline	Transfer Learning	Difference
mAP50 Mean	0.0552	0.5525	+0.4973
mAP50 Std Dev	0.0225	0.0870	+0.0645
Relative Gain	-	-	+900.38%
Paired t-test p-value	-	-	8.26e-05

5.4 Comparative Visualizations

To clearly illustrate the performance gap between the two experimental groups, the final step of the evaluation pipeline generated two comparative charts using the seaborn and matplotlib libraries. These visuals were built directly from the CSV metric data extracted in the previous phase.

The first chart, Figure 4, provides a fold by fold breakdown of the mAP50 scores. By plotting the scratch baseline directly alongside the transfer learning model for each of the five cross validation splits, the consistency of the domain adaptation strategy becomes very obvious. While the bars for the scratch baseline are barely visible across all five folds, the transfer learning model consistently peaks above 0.40. This graph reinforces the conclusion from the statistical testing, visually proving that the pre-trained weights did not just get lucky on one specific data split. Instead, the lunar initialization provided a massive and reliable advantage across the entire Martian dataset.

The second chart, Figure 5, shifts from a fold specific view to an overall summary of all four tracked metrics: Precision, Recall, mAP50, and mAP50-95. For this plot, the data from all folds was averaged out, and standard deviation error bars were added to the top of each column. Looking at this summary chart, the most striking visual difference is the massive gap in Recall between the two models. The error bars also tell a very important story about model stability. They visually confirm that the transfer learning approach handles the low data regime much better than the scratch baseline, remaining highly stable across all four measurement categories.

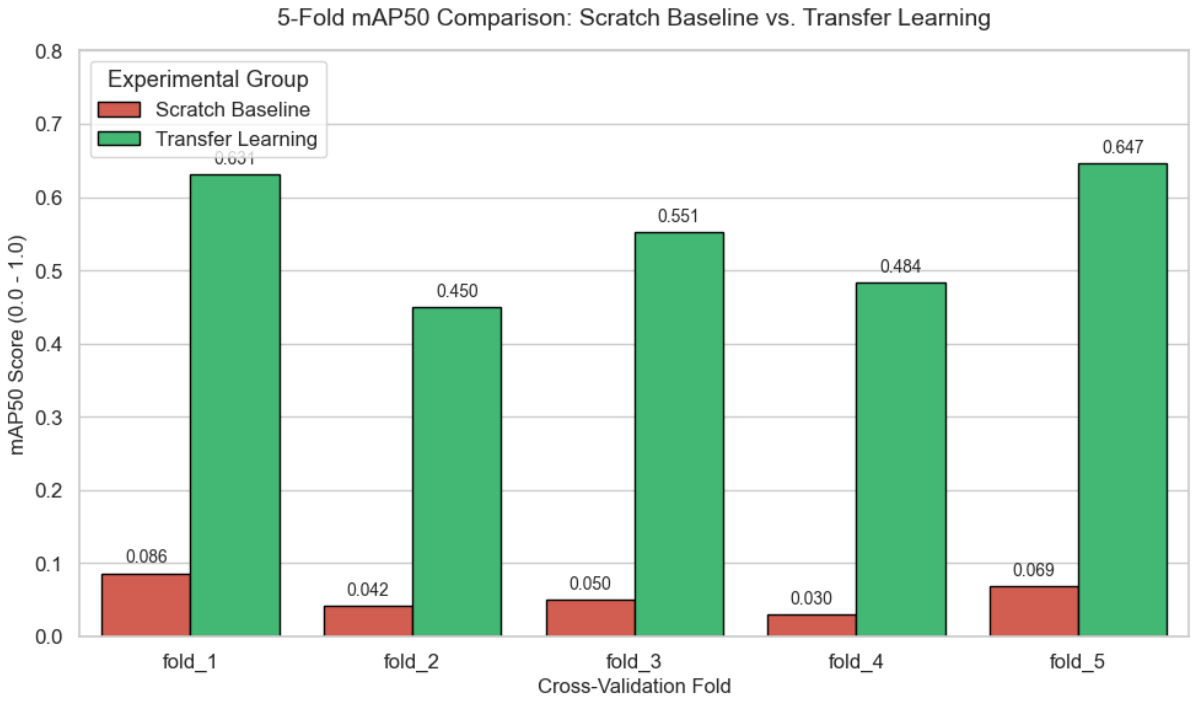


Figure 4 - A bar chart comparing the mAP50 performance of the scratch baseline against the transfer learning model across all five individual validation folds.

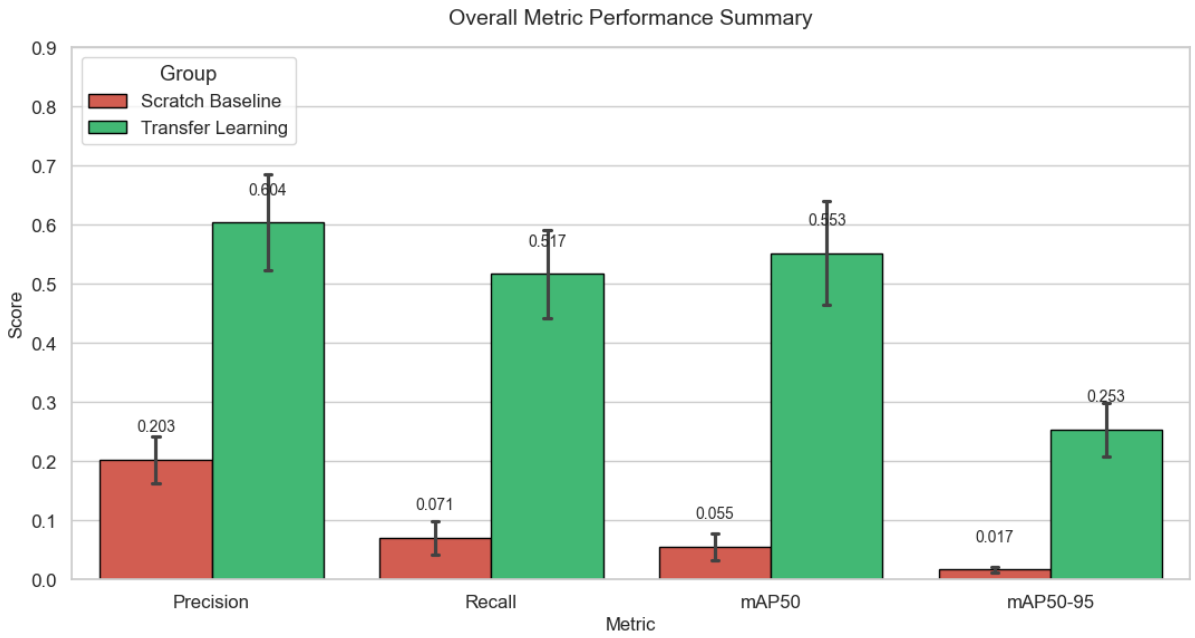


Figure 5 - An aggregated summary chart showing the mean scores for Precision, Recall, mAP50, and mAP50-95, complete with standard deviation error bars.

Chapter 6: Critical Discussion and Error Analysis

6.1 Analysis of Domain Adaptation Efficacy

Looking back at the primary research question, the goal of this project was to determine if pre-trained lunar weights could bypass the massive data requirements normally needed to train a crater detection model for Mars. Based on the quantitative results, the domain adaptation strategy was very effective, but it is still important to critically evaluate why it worked and where its limits lie.

The poor performance of the scratch baseline was completely expected. The YOLOv8s architecture contains over 11 million internal parameters. Trying to optimize that many variables with a training split of only 95 images is almost impossible. The network simply lacked the training volume needed to learn general crater features before it began overfitting on the small dataset. This explains why the recall was so low, as the model essentially failed to recognize the underlying terrain features.

In contrast, the cross-planetary transfer learning model showed that low-level visual features transfer well between celestial bodies. By pre-training on the large lunar dataset, the network had already calibrated its early convolutional layers to detect circular shadows, curved rims, and impact geometries. This behaviour aligns directly with established neural network layer dynamics theory, which demonstrates that early convolutional layers function as universal feature detectors that transfer effectively across differing domains [7], [8]. When exposed to the Martian folds, it did not have to learn what a crater looked like from scratch. It only needed the 95 training images to adjust its existing weights to the specific visual conditions of Mars, such as softer edges caused by wind erosion and atmospheric haze.

However, a realistic evaluation requires looking at the limits of this performance. While jumping from 5.5% to 55.2% mAP50 is a huge relative improvement, an absolute score around 55% means the model still misses a noticeable number of craters and sometimes struggles to place tight bounding boxes around them. This shows that while cross-planetary adaptation helps bridge the data gap, it cannot remove the domain shift entirely. The specific geological conditions on Mars, where older craters are often filled with sand or worn down by dust storms, still create visual differences that lunar data cannot fully account for. To reach an accuracy level suitable for real-time spacecraft navigation, the pipeline would likely need a slightly larger Martian dataset or targeted data augmentation techniques designed to simulate Martian surface conditions. This suggests that to push the accuracy into a range suitable for real-time spacecraft deployment, the pipeline would likely need a slightly larger Martian dataset or more advanced generative data augmentation techniques designed to simulate Martian atmospheric interference [8], [9].

6.2 Qualitative Evaluation of Target Detections

To fully understand the numerical results from Chapter 5, it helps to look at how the models actually behaved when presented with raw Martian imagery. To do this, a visual evaluation script was added to the end of the pipeline. This script loaded the saved weights from the first cross-validation fold for both the scratch and transfer learning models. It then picked a few random

validation images and ran side-by-side inferences at a confidence threshold of 0.25, plotting the predictions next to the original image for a direct qualitative comparison.

When reviewing the visual outputs, the predictions match exactly what the statistical metrics suggested. The scratch baseline model visibly struggled to understand the terrain. In most of the generated grids, the scratch model completely failed to place bounding boxes over clear, obvious crater structures, which perfectly explains its incredibly low recall scores. On the rare occasions it did manage to find a crater, the bounding boxes were often poorly scaled, either capturing too much background terrain or cutting off the edges of the crater rim. This visual evidence confirms that 95 images are simply not enough for the network to learn how to frame an object properly.

On the other hand, the cross-planetary transfer learning model showed a much deeper understanding of the Martian landscape. The side-by-side grids reveal that this model successfully detected almost all the prominent craters in the validation images. More importantly, the bounding boxes predicted by the transfer model were significantly tighter and more aligned with the actual crater boundaries. Because the early layers of the network already knew how to look for circular edges and shadow gradients from the lunar pre-training, the model was able to focus entirely on framing the structures accurately.

However, examining the images also reveals where the transfer learning model still falls short, which makes sense given its overall accuracy was around 55%. While it easily detected clear, isolated craters, the visual maps show that the model sometimes struggled with heavily eroded craters or instances where multiple impact zones overlapped each other. It also occasionally placed false positive bounding boxes over circular sand dunes or shadowed wind streaks that mimicked crater rims. These specific visual errors highlight the ongoing challenge of the domain shift, as the unique weather patterns on Mars can still occasionally trick a network that was primarily trained on lunar impacts.

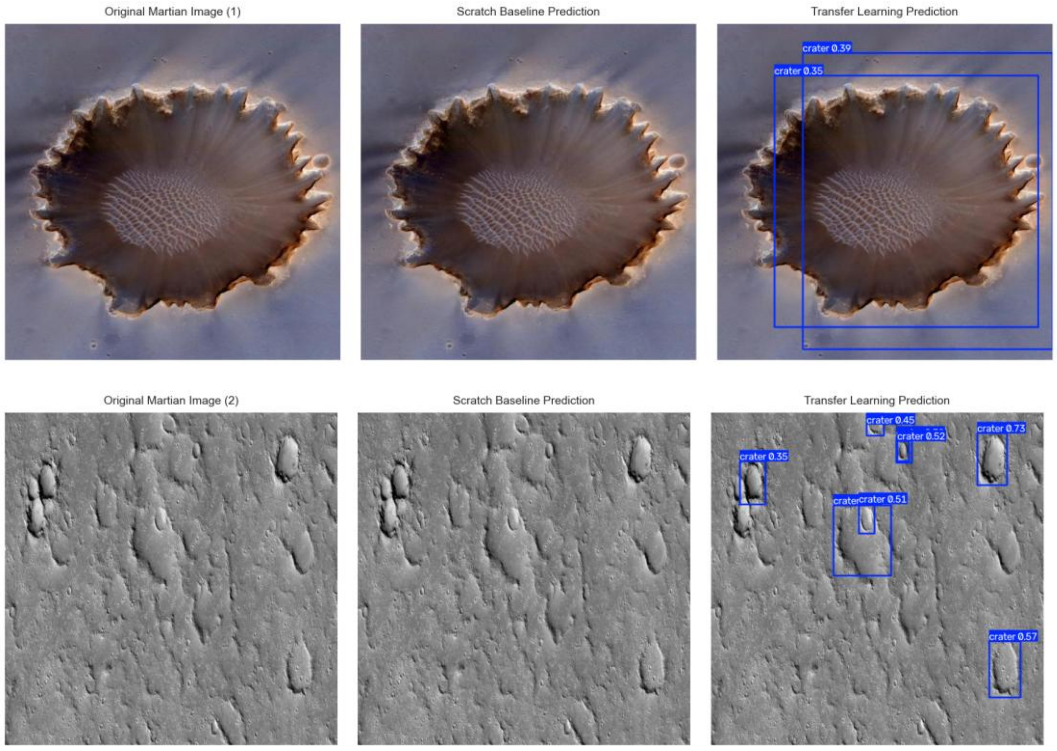




Figure 6 - A side-by-side visual comparisons showing the original Martian validation images alongside the predictions made by the scratch baseline and the transfer learning model.

6.3 System Limitations

The most obvious limitation of the system is the sheer size of the target dataset. Although restricting the Martian dataset to exactly 119 images was an intentional choice to simulate a strict low data regime, it naturally caps how well the model can learn. The 95 training images available per fold simply cannot capture the full geological diversity of the entire Martian surface. Because the model only saw a tiny fraction of what Mars looks like, its ability to generalize to completely unseen regions or highly unusual crater formations remains somewhat limited.

Another technical limitation is the training configuration. Due to time and resource constraints within the Colab environment, the models were only trained for 30 epochs using the default YOLO auto-optimizer settings. In a full production scenario, a network would normally be trained for hundreds of epochs accompanied by a systematic grid search to find the optimal learning rate, batch size, and weight decay for the target data. Stopping at 30 epochs means the transfer learning model might not have fully converged, leaving some potential accuracy on the table. Stopping at 30 epochs means the transfer learning model might not have fully converged, reflecting common resource and training constraints found in rapid prototyping environments [15].

Additionally, the experiment was restricted to a single architecture size, specifically the small version of YOLOv8. While the small variant is excellent for fast processing and avoiding VRAM memory crashes in Colab, it has fewer layers and parameters than the medium or large versions. A deeper and more complex network might have possessed the mathematical capacity needed to better distinguish between genuine crater rims and the confusing shadows caused by Martian sand dunes.

Chapter 7: Conclusion & Future Work

7.1 Summary of Research Findings

This dissertation set out to address a major bottleneck in automated planetary mapping: the severe lack of labeled training data for Martian orbital imagery. The primary objective was to determine if cross-planetary transfer learning could overcome this data scarcity by borrowing structural knowledge from a heavily mapped celestial body like the Moon.

The experimental results provided a definitive answer. When attempting to train a standard object detection model entirely from scratch using only 95 Martian images per training fold, the network completely failed to generalize. It achieved a mean mAP50 of just 5.52%, confirming the initial assumption that modern deep learning architectures cannot learn complex geological features from random initialization under extreme low-data conditions.

However, the domain adaptation pipeline proved to be highly effective. By first pre-training the YOLOv8 architecture on over 10,000 high-resolution lunar images, the model developed a strong foundational understanding of fundamental crater geometry. When this pre-trained model was fine-tuned on the exact same restricted Martian dataset, the mean mAP50 jumped to 55.25%. The model's ability to recall craters increased dramatically, and its bounding box predictions became significantly more accurate.

Formal statistical testing confirmed that this relative performance gain was highly significant and not simply the result of a lucky data split. Ultimately, this research proves that basic shapes like crater rims and shadows translate really well from one planet to another. Reusing a data-heavy world like the Moon is a genuinely practical way around data shortages on other planets.

7.2 Thesis Academic Contributions

This project brings a few clear contributions to the table for both planetary science and applied computer vision. One of the main things it does is set up a practical benchmark for cross-planetary transfer learning. A lot of existing research either focuses on general domain adaptation or relies on models pre-trained on everyday Earth images. This dissertation, however, provides direct proof that feature extractors trained specifically on lunar craters can be successfully repurposed for the Martian surface.

Beyond just swapping the model weights, this research also validates how well deep learning can survive under extreme data scarcity. By proving that a pre-trained network can reach a 55 percent mAP50 score using only 95 training images per fold, the project shows that you do not actually need thousands of labeled target images to get a functional detector running on a new planet. This is an important takeaway for future robotic missions, as it offers a realistic way to map celestial bodies where we simply do not have much visual data to work with yet.

Finally, the project provides a highly reliable and reproducible validation framework. By building a custom 5-fold cross validation harness and pairing it with formal hypothesis testing, the evaluation moves past the basic static data splits often seen in early machine learning projects. This setup guarantees that the performance boosts reported in this study are genuine and not

just the result of a lucky data split. Ultimately, this evaluation framework can easily be picked up and reused by other researchers for future planetary mapping experiments.

7.3 Future Research Trajectories

Building on what was learned in this project, there are a few really interesting directions future research could take to improve automated crater detection even more. One clear next step is looking into generative domain adaptation. Instead of just transferring weights, researchers could use Generative Adversarial Networks or diffusion models to artificially add Martian dust and atmospheric haze to high resolution lunar images. Training a model on a massive dataset of these synthetic images would help the network bridge the visual gap before it even sees a single real Martian image. Training on high volumes of photo-realistic synthetic images would retain the geometric diversity of the Moon while bridging the visual domain gap prior to fine-tuning on real Martian data [8].

Another area to explore is the model architecture itself. This project stuck to the small version of YOLOv8 so that training times and memory usage stayed manageable within the Google Colab environment. Future work should definitely test out larger, more complex models like YOLOv8x or Vision Transformers to see if the extra parameters help distinguish tricky features like overlapping craters. It would also be highly beneficial to run a formal ablation study to test exactly which layers of the network should be frozen and which should be updated during the transfer learning phase.

Relying only on optical imagery is always going to be difficult because of changing sun angles and shadows. A great way to fix this in the future would be to combine standard visual camera feeds with elevation data, like the topographical maps generated by the Mars Orbiter Laser Altimeter. If a network could look at both the physical height changes and the visual shadows at the same time, it would drastically cut down on the false positives caused by flat, circular sand dunes [3, 16].

Finally, if these models are actually going to be used on autonomous rovers or landers, they need to be tested on spaceflight hardware. Future engineering work should look into quantizing these networks down to 8-bit precision so they can run smoothly on power constrained, radiation hardened computers. Testing the inference speeds on embedded platforms like NVIDIA Jetson boards would be the final practical step to turn this research into a fully operational navigation tool for deep space exploration.

Reflective Essay

Reflecting on this Master's project, the journey from initial conceptualization to the final evaluation of my cross-planetary transfer learning pipeline has been both a technical milestone and a personal achievement. I have always had a strong interest in space and exploration, and I wanted my dissertation to reflect that. Working on real astronomical datasets and tackling space-focused computer vision felt like a meaningful way to connect my personal interests with my academic studies.

Finding unique datasets for this project at the start was genuinely one of the biggest hurdles. Because I was already comfortable and well-equipped with pulling and working with datasets from Kaggle, I figured sourcing them would be a quick checkbox task. I originally thought that by streamlining the data acquisition process, I would be able to save time and invest more energy into other technical aspects of the project, like tuning the model and handling the cross-validation logic. However, dealing with data contamination, such as accidentally mixing lunar and Martian images in the downloaded repositories, reminded me that data engineering is rarely straightforward.

Exploring whether a YOLOv8 object detection model pre-trained on lunar topography could generalize to a data-scarce Martian environment under a severe low-data regime was a great challenge. Building automated dataset extraction scripts and engineering a custom 5-fold cross-validation harness gave me practical experience in machine learning engineering. More importantly, it served as a fantastic first stepping stone into understanding the intersection of artificial intelligence and space technology, showing me how deep learning can address real data scarcity problems in planetary science.

What made this project enjoyable was working on something I genuinely cared about. While there were frustrating moments, such as dealing with Google colab environment limits and debugging training scripts, staying motivated was easy because the topic genuinely interested me. Seeing the transfer learning model jump to a 55.25% mAP50 score and analyzing where Martian dust storms or terrain features caused errors made the hard work worthwhile.

Ultimately, this project has been a rewarding conclusion to my Master's degree. It improved my programming and analytical skills, validated my approach to handling low-data regimes, and confirmed that I want to continue working on applied AI projects within the aerospace and space exploration sector.

AI Statement

I declare that this dissertation is entirely my own original work and that I am the sole author of the text, engineering implementation, and data analysis contained herein.

In accordance with QMUL's academic integrity guidelines, generative AI tools were utilized during the preparation of this report exclusively for the purposes of structural organization, document formatting optimization and the arrangement of data tables and visual layouts to enhance readability. The AI was not used to synthesize research findings, generate empirical results or substitute for independent academic writing and critical analysis.

References

[1] M. Sinha and S. Paul, "Automated detection of craters on the lunar surface using deep learning: A review with insights from Chandrayaan-2 TMC-2 data," Dec. 2025.

Available: https://www.sciencedirect.com/science/article/pii/S2211714825000366

[2] L. F. Palafox, C. W. Hamilton, S. P. Scheidt, and A. M. Alvarez, "Automated detection of volcanic landforms on Mars using convolutional neural networks," *Computers & Geosciences*, vol. 101, pp. 48–56, Apr. 2017.

Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC5701651/

[3] L. M. Downes, T. J. Steiner, and J. P. How, "Deep Learning Crater Detection for Lunar Terrain Relative Navigation," in *Proc. AIAA SciTech Forum*, Jan. 2020.

Available: https://dspace.mit.edu/entities/publication/db0f7436-589c-4cb3-bfce-19b7d1e5fe99

[4] M. Finkelstein, S. M. Baby, and H. Kitano, "Automatic Lunar Crater Detection from Optical Images and Elevation Maps," Stanford University, Tech. Rep. CS229, 2019.

Available: https://cs229.stanford.edu/proj2019aut/data/assignment_308832_raw/26614647.pdf

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

Available: https://arxiv.org/abs/1506.02640

[6] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2009.

Available: https://www.cse.ust.hk/~qyang/Docs/2009/tkde_transfer_learning.pdf

[7] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 27, 2014, pp. 3320–3328.

Available: https://arxiv.org/abs/1411.1792

[8] Y. Liu et al., "Two-Stage Adaptive Network for Semi-Supervised Cross-Domain Crater Detection under Varying Scenario Distributions," *Remote Sens.*, vol. 16, no. 11, p. 2024, May 2024.

Available: https://www.mdpi.com/2072-4292/16/11/2024

[9] L. Mu et al., "YOLO-Crater Model for Small Crater Detection," *Remote Sens.*, vol. 15, no. 20, p. 5040, Oct. 2023.

Available: https://www.mdpi.com/2072-4292/15/20/5040

[10] A. Silburt et al., "Lunar crater identification via deep learning," *Icarus*, vol. 317, pp. 27–38, Jan. 2019.

Available: https://arxiv.org/abs/1803.02192

[11] G. C. Cawley and N. L. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *J. Mach. Learn. Res.*, vol. 11, pp. 2079–2107, Jul. 2010.

Available: https://jmlr.org/papers/volume11/cawley10a/cawley10a.pdf

[12] T. Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.

Available: https://arxiv.org/abs/1405.0312

[13] J. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, Jun. 2010.

Available:https://www.researchgate.net/publication/220659463_The_Pascal_Visual_Object_Classes_VOC_challenge

[14] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Nov. 2011.

Available: https://jmlr.org/papers/v12/pedregosa11a.html

[15] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

Available: https://arxiv.org/abs/1912.01703

[16] X. Zhang, Y. Wang, and L. Wang, "DBYOLO: Dual-Backbone YOLO Network for Lunar Crater Detection," Remote Sensing, vol. 17, no. 19, p. 3377, Oct. 2025.

Available: https://www.mdpi.com/2072-4292/17/19/3377

[17] R. La Grassa, "LU3M6TGT: Lunar Crater Dataset for Object Detection," Kaggle Repository, 2022.

Available: https://www.kaggle.com/datasets/riccardolagrassa/lu3m6tgt

[18] Z. Lincoln, "Martian/Lunar Crater Detection Dataset," Kaggle Repository, 2021.

Available: https://www.kaggle.com/datasets/lincolnzh/martianlunar-crater-detection-dataset

[19] A. Paszke et al., "Automatic Crater Detection Using Convex Grouping and Convolutional Neural Networks," Progress in Artificial Intelligence, vol. 9, no. 2, pp. 105–118, Jun. 2020.

Available: https://www.researchgate.net/publication/300124757_Automatic_Crater_Detection_Using_Convex_Grouping_and_Convolutional_Neural_Networks

[20] S. S. Bhunia et al., "Automated Lunar Surface Image Classification Using Deep Convolutional Neural Networks for Geological Feature Detection," *Am. J. Neural Netw. Appl.*, vol. 10, no. 2, pp. 34–45, Oct. 2024.

Available: https://www.sciencepublishinggroup.com/article/10.11648/j.ajnna.20241002.11

[21] V. Landers, S. Rossi, and M. Schulz, "Vision-Language Model for Accurate Crater Detection under Parameter-Efficient Fine-Tuning," arXiv preprint arXiv:2601.07795, Jan. 2026.

Available: https://arxiv.org/abs/2601.07795